Prerequisites and objectives

- Prerequisites are BSTs, binary search, ordered sequences, tree height, and disk-page concepts.
- Derive B-tree occupancy and height constraints from multiway searching. Trace splits, borrowing, and merging.
- Distinguish B-tree and B+ tree record placement, separator semantics, and range-scan costs.

Multiway search intervals

- An internal node stores increasing keys $k_1 < \dots < k_r$, at most $m-1$ keys, and $r+1$ child subtrees.
- Child intervals are $(-\infty, k_1)$, (k_1, k_2), …, $(k_r, +\infty)$, further restricted by ancestor intervals.
- Binary search within one node uses O(log m) CPU comparisons. An arbitrary m-way tree guarantees neither balance nor minimum occupancy.
- If nodes may retain only one child or leaves have unequal depths, maximum fanout m alone does not imply height $O(\log_m n)$.

External memory and fanout

- Data is divided into blocks. One I/O transfers one block between main and external memory. B denotes the number of items fitting a block.
- Putting multiple keys and child pointers on one page allows one page read to choose the next-level path. Larger fanout reduces levels.
- With internal and leaf capacities Θ(B), $O(1 + \log_B n)$ counts page I/Os. It does not mean O(1) CPU comparisons within every page.
- Page size, key length, pointers, headers, and fill ratio determine actual fanout. SSD pages and caches also matter, but theoretical bounds alone do not determine latency.

B-tree minimum degree t

- Use integer $t \geq 2$. Every nonroot node has $t-1$ through $2t-1$ keys. An internal node has one more child than keys.
- A nonleaf root has at least one key and two children. A leaf root may be empty. All leaves have the same depth.
- Keys and subtree intervals obey search order. Records may be stored in internal nodes as well as leaves.
- Another text's order m may mean maximum child count. Convert definitions before comparing formulas. It is not automatically our t.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Compare capacities before algorithms

Here **order m** means at most m children per node. A non-root internal node has $\lceil m/2 \rceil$ through m children and one fewer key; roots and leaves have their own rules.

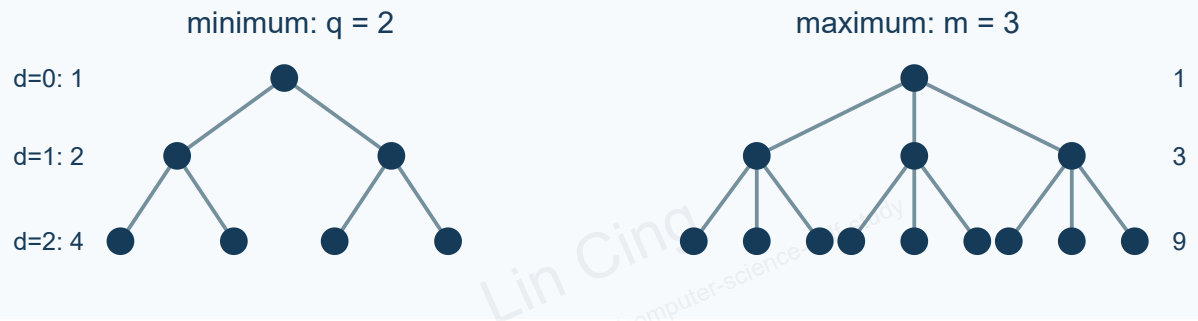
Convention	Non-root children	Non-root keys	Example
order m	$\lceil m/2 \rceil$ to m	$\lceil m/2 \rceil - 1$ to $m-1$	m=3: 1–2 keys
minimum degree t	t to 2t	$t-1$ to $2t-1$	t=2: 1–3 keys

When $m=2t$, both occupancy bounds match. For odd m no integer t matches both limits. An order-3 2–3 tree cannot use $t=1.5$ because this chapter's t algorithm requires integer $t \geq 2$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Node bounds for a general order m

Let q=⌈m/2⌉ and count nonempty-tree height h in edges from root to leaf. The root has at least one key; when h≥1 it has at least two children, and each non-root internal node has at least q children. The minimum numbers of nodes by depth are 1, 2, 2q, …, $2q^{h-1}$.



The minimum number of nodes is $1 + 2(1 + q + \cdots + q^{h-1}) = 1 + 2(q^h - 1)/(q - 1)$. Each non-root node has at least q−1 keys, giving $n \geq 2q^h - 1$. When h=0 the nonempty root is a leaf and n≥1.

Maximum capacity and a 2–3 tree example

Each level has at most m times as many nodes as the previous level. If every node has m−1 keys, the maximum number of nodes at height h is $(m^{h+1} - 1)/(m - 1)$ and the maximum number of keys is $m^{h+1} - 1$.

Order	h (edges)	Min nodes	Max nodes	Min keys	Max keys
3	2	7	13	7	26
4	2	7	21	7	63

For order 3 and h=2, a minimum tree has 1 root, 2 nodes at the next depth, and 4 leaves. Each has one key, totaling 7. A maximum tree has 1, 3, 9 nodes by depth, each with two keys, totaling 26.

If height H counts levels, H=h+1. To infer height from n, first filter integer h using $2q^h - 1 \leq n \leq m^{h+1} - 1$, then check a concrete construction.

Why an order-3 node splits after overflow

A non-root 2–3 tree node holds one or two keys. Splitting a full two-key node before insertion would promote one key and leave only one for two children, which both need at least one key.

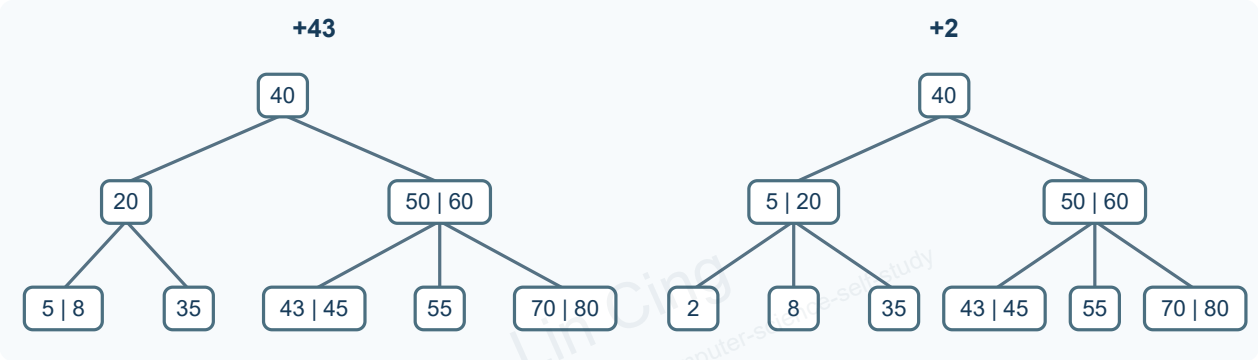
Insert first, temporarily creating three keys. Promote the middle key and keep one on each side. If the parent now has three keys, split it in turn. Only a root split increases tree height.

The top-down t=2 algorithm is a different complete procedure: its full node has three keys, so it can promote one and leave one on each side before descent.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

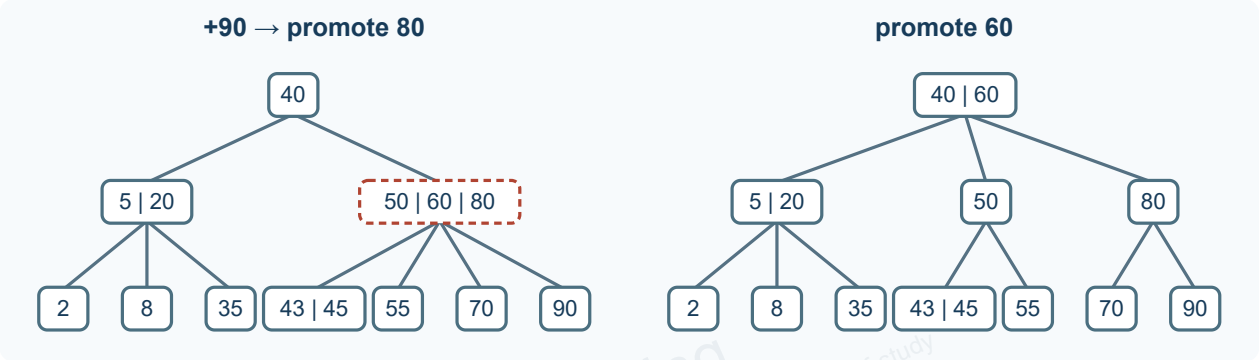
A 2–3 tree insertion trace: 43 and 2

Initially root [40] has internal children [20] and [50,60]. Their leaves are [5,8], [35], [45], [55], [70,80] from left to right. Insert 43, 2, then 90.



Inserting 43 changes [45] to [43,45]. Inserting 2 temporarily changes [5,8] to [2,5,8]; promote 5 into [20], leaving leaves [2] and [8]. In this B-tree the promoted key moves out of the leaf.

Insert 90: a split propagates upward



The leaf [70,80] becomes [70,80,90]. Promote 80, temporarily making its parent [50,60,80]. Promote 60 to the root; new internal nodes [50] and [80] each retain their two corresponding children. The root becomes [40,60].

Every non-root node now has one or two keys, and all leaves have equal depth. Only 43, 2, and 90 were added. Promotion moves a B-tree data key; it does not duplicate a record in the parent.

Deriving the B-tree height bound

- For a nonempty tree of edge height h≥1, the root has at least two children and subsequent nonroot internal nodes at least t children.
- Depth d≥1 contains at least $2t^{d-1}$ nodes, each with at least t−1 keys.
- Therefore $n \geq 1 + 2(t-1)\sum_{d=1}^{h} t^{d-1} = 2t^h - 1$, giving $h \leq \log_t((n+1)/2)$.
- If t=Θ(B) and one node fits one block, search follows one root-to-leaf path in $O(\log_B n)$ I/Os for nonsmall trees. Caching the root can save one read. A root-only tree needs O(1) I/Os.

Search checks both keys and child intervals

1. Find the first node key not less than x, at position i.
2. On equality, return the record. If not equal and at a leaf, report absence.
3. Otherwise descend into child i and repeat.

Root keys `[20,50]` direct query 37 to the middle child. A B-tree search can succeed at an internal key. Our B+ search must continue to a leaf.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Split a full child before descending

- Use top-down insertion. If the root is full, create a new root and split the old root before descent.
- A full node contains 2t−1 keys. Promote its tth key to the parent, leaving t−1 keys on each side with corresponding children.
- Split full children before entering them, so the eventual leaf has space. The parent also has space by the descent invariant.
- Splitting preserves the key set and equal leaf depth. Only a root split increases overall height.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Worked t=2 insertion trace

- Insert `10,20,5`, giving full leaf root `[5,10,20]`.
- Before inserting 6, split the root and promote 10. Leaves become `[5]` and `[20]`. Insert 6 into the left leaf to get `[5,6]`.
- Insert 12 and 30, filling the right leaf `[12,20,30]`. Before inserting 25, split it and promote 20.
- The final root is `[10,20]`, with leaves `[5,6]`, `[12]`, `[25,30]`. Internal keys 10 and 20 are not also retained in leaves.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete B-tree split and insertion

```
split_child(parent,i):
    y=parent.children[i]; require len(y.keys)==2*t-1
    z=Node(leaf=y.leaf,keys=[],children=[])
    median=y.keys[t-1]
    z.keys=y.keys[t:]; y.keys=y.keys[:t-1]
    if not y.leaf:
        z.children=y.children[t:]; y.children=y.children[:t]
    insert median at parent.keys[i]
    insert z at parent.children[i+1]
insert_nonfull(x,k):
    i=lower_bound(x.keys,k)
    if i<len(x.keys) and x.keys[i]==k: return already_present
    if x.leaf: insert k at i; return
    if x.children[i] is full:
        split_child(x,i)
        if k>x.keys[i]: i+=1
        else if k==x.keys[i]: return already_present
    insert_nonfull(x.children[i],k)
```

Complete B-tree split and insertion (continued)

If the root is full, first create a new root with the old root as its sole child, split_child(newroot,0), then call insert_nonfull. Internal splitting divides child pointers too: t children per side correspond to t−1 keys. Only a root split raises all leaf depths.

After the chapter sequence at t=2, root=[10,20], leaves=[5,6],[12],[25,30]. Insert 7 into [5,6,7]. Before inserting 8, split that leaf and promote 6: root=[6,10,20], leaves=[5],[7,8],[12],[25,30]. The parent is now full but this insertion is done; the next insertion splits the root before descending. Top-down full-node splitting is distinct from overflow-first splitting.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Strengthen a child before deletion descent

- If the destination child has only t−1 keys, borrow or merge first so deletion cannot violate minimum occupancy.
- If an adjacent sibling has at least t keys, rotate one key through the parent separator, moving a child pointer too when necessary.
- If adjacent children both have t−1 keys, merge them with their parent separator to obtain 2t−1 keys.
- Borrowing preserves height. Merging reduces the parent's key count. The top-down invariant keeps nonroot parents legal, with a separate root case.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Internal-key deletion and root contraction

- For an internal target key, if the left child has at least t keys, replace the target with its predecessor and delete that predecessor in the left subtree.
- Otherwise use the successor if the right child has at least t keys. If both have t−1 keys, merge them with the target and delete within the merged node.
- A root with zero keys and one child is replaced by that child, reducing height by one. An empty leaf root represents an empty tree.
- Only constantly many node reorganizations occur per level. I/O is $O(1 + \log_t n)$. With array movement of t keys per node, CPU work can be $O(t(1 + \log_t n))$. The additive constant covers a root-only tree.

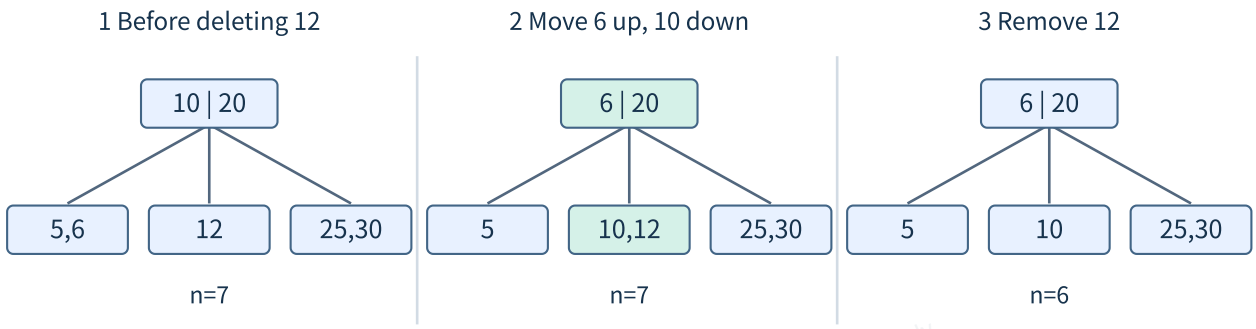
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Worked borrowing trace

- Delete 12 with t=2 from root `[10,20]` and leaves `[5,6]`, `[12]`, `[25,30]`.
- The middle leaf has only one key. Borrow from the left before descending. Parent key 10 moves down, and left-sibling maximum 6 moves up.
- The state becomes root `[6,20]`, leaves `[5]`, `[10,12]`, `[25,30]`. Delete 12, leaving middle leaf `[10]`.
- Every nonroot node has at least one key, all leaves have equal depth, and exactly 12 has left the key set.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

B-tree borrowing moves two data keys



With t=2, every nonroot leaf needs 1 to 3 keys. Before descending into [12], move the parent's data key 10 into the middle leaf and the left sibling's maximum data key 6 into the vacated parent slot. Two distinct records move; 6 is not copied while also remaining in the left leaf. Only after state 2 has [10,12] may deletion remove 12, producing state 3.

Inorder is [5,6,10,12,20,25,30] in both first states, then [5,6,10,20,25,30]. Borrowing neither loses nor duplicates a record; deletion removes exactly one. All leaves remain at depth 1. An internal sibling also transfers its boundary child pointer, which the leaf picture does not show. A B+ parent separator is only a routing copy, so its leaf-borrowing rule must not be replaced by this B-tree data movement.

B-tree deletion borrowing, merging, and root contraction

```
prepare_child(x,i):
    child=x.children[i]
    if len(child.keys)>=t: return i
    if i>0 and len(x.children[i-1].keys)>=t:
        left=x.children[i-1]
        child.keys.insert(0,x.keys[i-1]); x.keys[i-1]=left.keys.pop()
        if not child.leaf: child.children.insert(0,left.children.pop())
    else if i+1<len(x.children) and len(x.children[i+1].keys)>=t:
        right=x.children[i+1]
        child.keys.append(x.keys[i]); x.keys[i]=right.keys.pop(0)
        if not child.leaf: child.children.append(right.children.pop(0))
    else:
        if i>0: merge_children(x,i-1); return i-1
        merge_children(x,i); return i
    return i
```

Delete directly at a leaf. At an internal target, use a predecessor if the left child has at least t keys, otherwise a successor if the right has at least t; if both have t−1, merge them with the target before descent. For an absent local target, prepare the destination and use its returned index, since merging left shifts i. Contract an empty root with one child.

B-tree deletion borrowing, merging, and root contraction (continued)

Deleting 12 first borrows 6 upward from [5,6], moving 10 downward. The result has root [6,20] and leaves [5],[10],[25,30]. To delete 5 next, merge [5]+6+[10] into [5,6,10], leaving root [20], then remove 5 to obtain [6,10]. Delete 25 from the right leaf to obtain [30], and 6 from the left to obtain [10]. Before deleting 10, merge [10]+20+[30] into [10,20,30]. The root empties and contracts to that leaf. Removal leaves [20,30].

Borrowing preserves order and leaf depth. Merging produces exactly2t−1 keys. One-page nodes give constant I/O per level and $O(1+\log_t n)$ total, while intra-page shifts may cost O(t) per level.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

B-tree root entry and complete deletion recursion

```
insert(k):
    if root is full:
        old=root; root=Node(leaf=false,keys=[],children=[old])
        split_child(root,0)
    insert_nonfull(root,k)
erase_from(x,k):
    i=lower_bound(x.keys,k)
    if i<len(x.keys) and x.keys[i]==k:
        if x.leaf: remove x.keys[i]; return
        if len(x.children[i].keys)>=t:
            p=maximum_key(x.children[i]); x.keys[i]=p
            erase_from(x.children[i],p)
        else if len(x.children[i+1].keys)>=t:
            s=minimum_key(x.children[i+1]); x.keys[i]=s
            erase_from(x.children[i+1],s)
```

B-tree root entry and complete deletion recursion (continued)

```
        else:
            merge_children(x,i); erase_from(x.children[i],k)
    else:
        if x.leaf: return absent
        i=prepare_child(x,i); erase_from(x.children[i],k)
erase(k):
    erase_from(root,k)
    if len(root.keys)==0 and not root.leaf:
        old=root; root=old.children[0]; release(old)
merge_children(x,i):
    left=x.children[i]; right=x.children[i+1]
    left.keys+= [x.keys[i]] + right.keys
    if not left.leaf: left.children+=right.children
    remove x.keys[i] and x.children[i+1]; release(right)
```

An empty B-tree has a zero-key leaf root, so erasing from it reports absence. Keys abbreviate complete key-value records; move the complete pair in an implementation. Merging two children with t−1 keys yields (t−1)+1+(t−1)=2t−1 keys, exactly within capacity. The intervening parent key is a real record, unlike a B+ routing separator.

B-tree root entry and complete deletion recursion (continued)

The descent invariant gives each nonroot recursive entry at least t keys. Prepare_child borrows or merges before entering it. An internal-key predecessor/successor branch chooses a child with at least t keys, leaving at least t−1 after removal. If neither can lend, merge both children with the target key first. A left merge must return i−1 because the old child[i] was released. An absent-key deletion may still restructure the search path; the abstract key set remains unchanged, while physical shape need not.

For root [20] and leaves [10],[30] with t=2, deleting 10 first merges both children and parent record 20 into [10,20,30]. Deletion leaves [20,30], and the empty internal root contracts to that leaf. Dropping separator 20 during merging would lose an undeleted record. Same-level sibling restructuring preserves equal leaf depth; root contraction changes the common depth for the entire tree.

B+ records stay in leaves

- All key-value records are in leaves. Internal nodes contain only separators and child pointers.
- A separator equals the minimum key of its right child subtree. On equality, descend right. Do not mix this with a left-maximum convention.
- Leaves are linked in key order, enabling sequential range scanning after locating the first leaf.
- Separators are routing information and can duplicate leaf keys. Counting internal keys plus leaf keys does not count data records.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Specify internal and leaf capacities separately

- Use integers $M \geq 3$, $L \geq 2$. Nonroot internal nodes have a through M children, where $a = \lceil M/2 \rceil \geq 2$. An internal root has 2 through M children. Internal key count is child count minus one. Minimum fanout at least two supports logarithmic height.
- Nonroot leaves contain $\lceil L/2 \rceil$ through L records. A leaf root may have 0 through L.
- M and L may differ. Internal entries store child pointers, while leaves may store record pointers or full records.
- After a leaf split, copy the right leaf's minimum into the parent. Internal splits promote a partition boundary and redistribute children. They do not use the same key-copy rule as leaf splits.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Worked B+ leaf split

- With L=3, inserting 10 into `[4,9,12]` gives temporary `[4,9,10,12]`.
- Split into `[4,9]` and `[10,12]`, copy separator 10 to the parent, and repair the leaf chain.
- If these are the only two children, root `[10]` routes a query for 10 rightward to the actual record.
- Unlike B-tree promotion, copying 10 into the parent does not remove it from its leaf.

Copy separator 10; record 10 remains in the right leaf



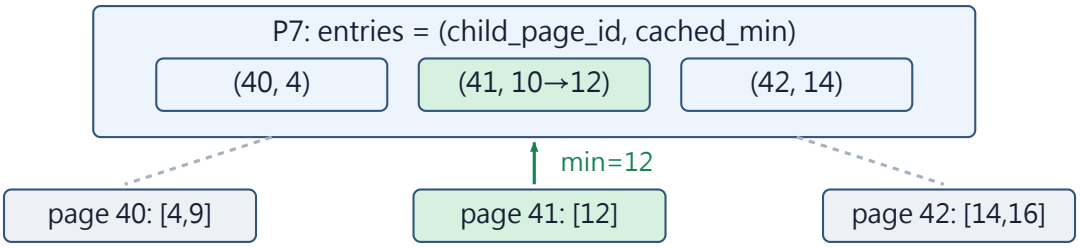
Disk-page layout and local minimum updates

```
// Internal page: entries[j]=(child_page_id,cached_min), all in this page.
// Separators are entries[1:].cached_min; no persistent parent pointers.
refresh_local(p):
    if p.leaf: p.minimum=p.records[0].key if nonempty else absent
    else: p.minimum=p.entries[0].cached_min
entry(p): return (p.id,p.minimum)
find_leaf(k):
    path=[]; p=read(root_id)
    while not p.leaf:
        i=upper_bound(p.entries[1:].cached_min,k)  // equality goes right
        path.push((p.id,i)); p=read(p.entries[i].child_page_id)
    return path,p
```

Each node occupies one disk page. An internal page stores every child page ID and an exact copy of that child's minimum inside the parent page. Separators are the cached minima after entry 0, so no child read is needed to obtain them. A leaf stores ordered records and next/prev page IDs. Changed children return new minima up an ancestor stack; unchanged entries retain their values.

Disk-page layout and local minimum updates (continued)

L=2: erasing 10 updates a minimum cached inside the parent page



Gray dashes are page-id links; green returns the already-read leaf minimum.
Read P7 and leaf 41 only; rebuild [12,14] from minima stored inside P7.

With M=3,L=2, page P7 initially stores (40,4),(41,10),(42,14), and leaf 41 holds [10,12]. Erasing 10 leaves [12], meeting minimum occupancy 1. Update entry 1 to (41,12), changing separators [10,14] to [12,14]. Pages 40,42 and their cached minima are unchanged and need not be read.

Reading M separate child.minimum fields can add M cold page reads to one refresh. Rewriting parents of Θ(M) children moved by an internal split likewise adds Θ(M) I/Os. Calling a field cached does not place it in the parent page. This layout explicitly places the copies locally and removes persistent parent fields. Store only (page_id,index) in the ancestor stack, rereading one parent on ascent.

Disk-page layout and local minimum updates (continued)

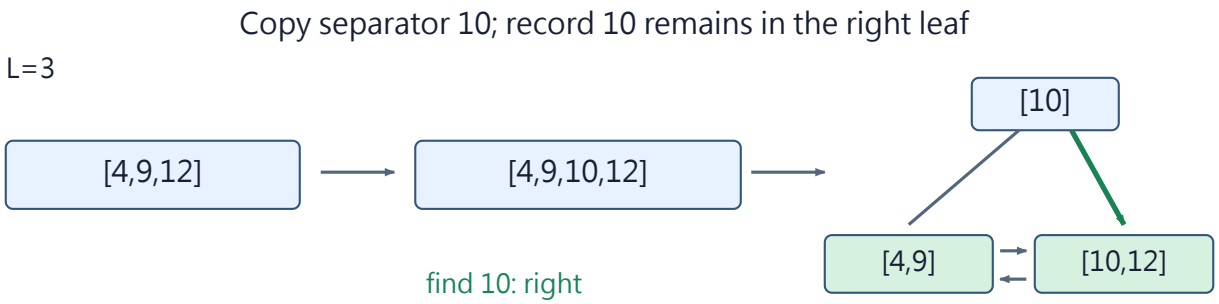
The reason for avoiding child-page parent rewrites also appears in Open Data Structures § 14.2.2.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

B+ insertion, exact separators, and upward splits

```
insert(k,v):
    path,node=find_leaf(k)            // stack holds (parent_page_id,index)
    insert/update (k,v) in sorted node.records
    while true:
        if node exceeds L records or M entries:
            parts=split_into_floor_ceil_halves(node)
            if node.leaf: splice both leaf links and repair adjacent leaf
        else: parts=[node]
        for p in parts: refresh_local(p); write(p)
        if path empty:
            if len(parts)==1: root_id=parts[0].id
            else:
                r=new_internal(entries=[entry(p) for p in parts])
                refresh_local(r); write(r); root_id=r.id
            return
        (parent_id,i)=path.pop(); parent=read(parent_id)
        parent.entries[i:i+1]=[entry(p) for p in parts]
        node=parent
// entry(p)=(p.id,p.minimum); internal splits move complete cached entries.
```


B+ insertion, exact separators, and upward splits (continued)



With L=3, insert 10 into [4,9,12], overflow to [4,9,10,12], then split into leaves [4,9],[10,12] under root [10]. Inserting 14 gives right leaf [10,12,14]. Inserting 16 splits it into [10,12], [14,16], and the root becomes [10,14]. The parent keys 10 and 14 are routing copies. Both records remain in leaves. Routing intervals are (−∞,10), [10,14), [14,+∞).

Use the preceding disk layout: each internal page stores (child_page_id,cached_min) entries locally; child pages have no parent field. The left half keeps the original page ID and the right gets a new page. Splitting moves complete entries with their cached minima. Refresh each new page from its own data and return only changed entries to the parent. Never fetch other child pages for their minima or rewrite moved children's on-disk parents.

B+ insertion, exact separators, and upward splits (continued)

Find_leaf saves ancestor page IDs and descent indices, rather than pinning all ancestor pages. On ascent reread one parent, replace the old entry with one or two returned entries, and split upward if needed. Only root overflow creates a new root. Propagate exact minima even without overflow. Nonroot internal fanout is at least a=ceil(M/2), leaf occupancy at least ell=ceil(L/2); $M \geq 3, L \geq 2$ make both split halves legal.

Each level touches constantly many pages. Doubly linked leaf splicing adds at most one adjacent-leaf page. Insertion therefore takes O(h+1) page I/Os in this layout, while intra-page array work is O(L+Mh) worst case. The ancestor stack stores O(h+1) words plus constantly many page buffers of O(M+L) words. Fix record, entry, and page sizes before translating a RAM representation into disk costs.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Deletion and boundary updates

- After leaf underflow, first try borrowing from an adjacent leaf. Otherwise merge and remove the parent's separator.
- Even without underflow, changing a right-subtree minimum requires updating the corresponding ancestor boundary under our exact-minimum convention.
- In a separate L=2 example, deleting 10 from right leaf `[10,12]` leaves `[12]`, which meets minimum occupancy 1. Still change the parent separator from 10 to 12.
- Some systems allow an older but still valid fence key. That convention differs. Exact right minima are incompatible with a claim that parent keys never need updating.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

B+ deletion borrowing, merging, and range retrieval

```
erase(k):
    path,node=find_leaf(k)
    if k absent: return
    remove record k from node
    while true:
        refresh_local(node)
        if path empty:
            if node internal and len(node.entries)==1:
                root_id=node.entries[0].child_page_id; free(node)
            else: write(node)
            return
        (parent_id,i)=path.pop(); parent=read(parent_id)
        if node meets its nonroot minimum occupancy:
            write(node); parent.entries[i]=entry(node)
        else:
```


B+ deletion borrowing, merging, and range retrieval (continued)

```
            read adjacent sibling(s) of the same parent
            if a sibling can lend:
                move a boundary record or complete cached entry
                refresh_local both; write both; update their parent entries
            else:
                merge adjacent records or cached-entry arrays
                repair both leaf links if leaves
                refresh_local/write merged page; free the other page
                replace their two parent entries by entry(merged)
        node=parent
range(lo,hi):
    locate first record>=lo
    scan leaf links until key>hi or leaf chain ends
```

Let L=3, with root [10,14], leaves [4,9],[10,12],[14,16], and minimum occupancy 2. Delete 10 to leave [12]. Neither sibling can lend, so merge with the left leaf into [4,9,12]. The root becomes [14], and leaf links join [4,9,12]↔[14,16]. Delete 14, leaving [16]. The left leaf can now lend 12, producing [4,9]↔[12,16] and root separator 12. Exact separators also update when a minimum changes without underflow.

B+ deletion borrowing, merging, and range retrieval (continued)

Borrow or merge complete local entries without reading moved child pages or changing persistent parent fields. After borrowing, continue minimum propagation through the saved stack even though underflow is repaired. Internal merging uses at most 2a−1≤M entries, leaf merging at most $2\ell - 1 \leq L$ records; same-level merging preserves leaf depth. Contract a one-child root by adopting its child page ID. Constantly many parent/sibling/neighbor-leaf page accesses per level give O(h+1) I/Os, with O(L+Mh) worst-case RAM shifts.

For range [9,14], locate 9, follow next to 12, and stop at 16>14, returning [9,12]. With general height h and minimum leaf occupancy ell, index I/Os are $O(h + 1 + k/\ell)$. Only M,L both proportional to block capacity B justify $O(1 + \log_B n + k/B)$. Full records stored elsewhere add D data-page reads, worst-case D=k. Output work remains Θ(k).

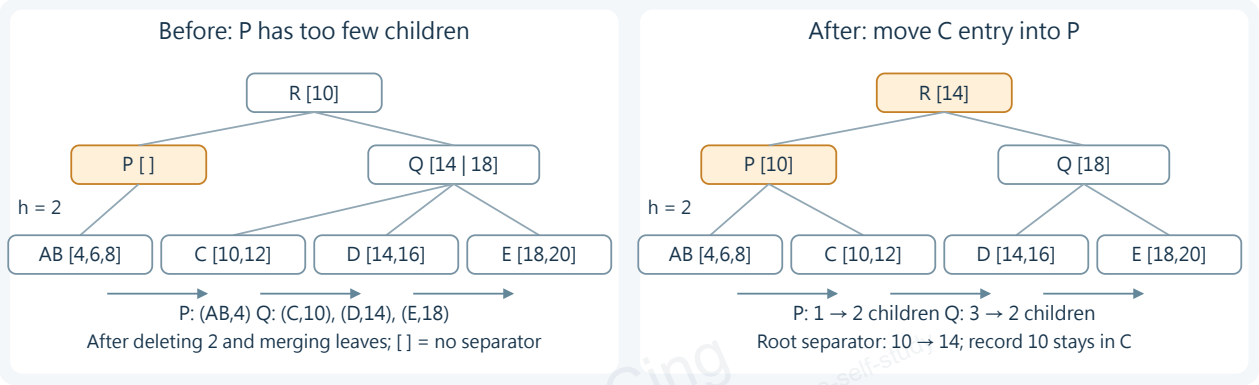
Internal underflow: count children, not just separators

Set M=4 and L=3. Each nonroot internal page needs at least two children; each nonroot leaf needs at least two records. Root R has children P,Q. P owns leaves A=[2,4], B=[6,8]; Q owns C=[10,12], D=[14,16], E=[18,20]. Every leaf is at depth two, and R has separator 10.

Step	Actual page changes	Why repair continues
Delete 2	A becomes [4]	One record is below the leaf minimum two
Inspect B	B=[6,8] cannot lend	Lending would leave B below minimum too
Merge A and B	Keep A's page ID as AB=[4,6,8]; free B	Three records fit L exactly
Update P	P.entries=[(AB,4)]; P.minimum=4	**One child is below the internal minimum two**

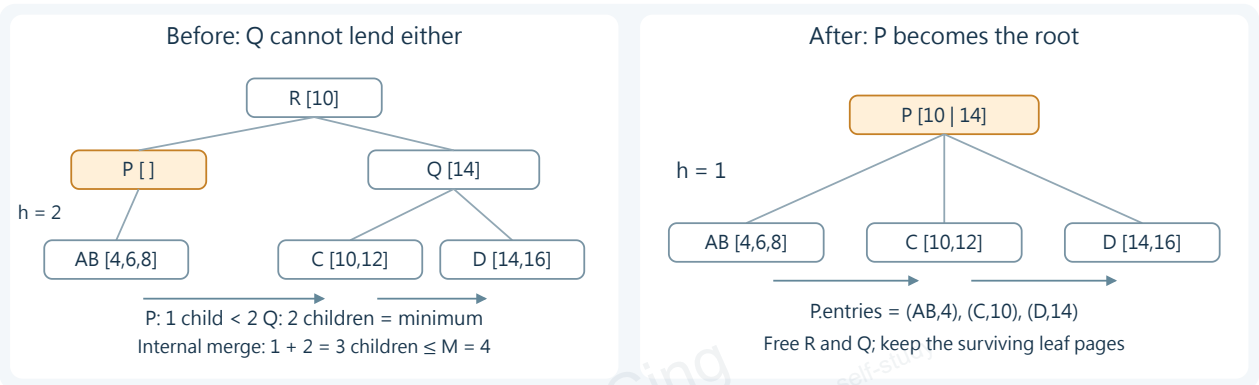
P has no separator but still points to leaf AB; it is not an empty leaf. Replace chain A↔B↔C with AB↔C and set C.prev to AB. The root temporarily holds (P,4),(Q,10). This is an intermediate update state; repair P before exposing a valid index.

Internal borrowing moves an entry, not a leaf record



Q has three children and can lend its first entry (C,10). Append it to P, giving P=[(AB,4),(C,10)] and Q=[(D,14),(E,18)]. Both now have two children. P's separator is 10; Q's is 18. Replace root entries (P,4),(Q,10) with (P,4),(Q,14).

When neither sibling can lend: merge and contract the root



Independently omit E, keeping A,B,C,D and the same capacities. Delete 2 and merge A/B: P=[(AB,4)], Q=[(C,10),(D,14)]. Q cannot lend. Concatenate into P=[(AB,4),(C,10),(D,14)], with separators [10,14], and free Q. R now has only (P,4): set root_id=P and free R. Height drops from two to one.

Check the rebuilt index through searches and the leaf chain

Each branch deletes only record 2. Internal repair moves page IDs and cached minima; C still physically stores [10,12]. The old parent separator 10 is not an extra record to insert into a leaf.

Check	After internal borrowing	After merging and root contraction
Internal pages and height	R=[14]; P=[10], Q=[18]; h=2	New root P=[10,14]; h=1
Search for 10	Left from R to P; equality goes right to C	Equality at P's 10 goes right to C
Leaf chain	AB↔C↔D↔E; nine records	AB↔C↔D; seven records
Record 10	Appears once, in C	Appears once, in C

In the contracted tree, range [8,15] first finds 8 in AB=[4,6,8]. Follow next to C=[10,12], then D=[14,16]. Emit 8,10,12,14 and stop upon inspecting 16>15. Diagram arrows show only next; prev points to the preceding page in the same chain. Internal repair does not change these leaf links.

Why internal repair preserves records, order, and leaf depth

Entries are ordered by subtree minima. Borrowing the right sibling's first entry shifts the boundary one subtree right; borrowing the left sibling's last entry does the reverse. Subtrees retain their contents and equal depth. Rebuild exact separators from the resulting entry arrays, and continue propagating changed minima into ancestor entries.

Write a=ceil(M/2). An underfull internal page has a−1 children; a sibling unable to lend has a. Their merge has 2a−1≤M children: M−1 for even M, M for odd M. Here 1+2=3≤4. **B+ merging concatenates child entries; it does not insert the parent separator as another data record.** The merged page automatically has one fewer separator than children.

At each level inspect the current page, its parent, and at most two adjacent siblings. Leaf merging additionally repairs constantly many chain fields. With no persistent child.parent field, moving an entry does not rewrite that child's page. The total is O(h+1) page I/Os; exact read/write counts still depend on buffer residency. Root contraction alone reduces every leaf's depth together by one.

B+ height needs fanout and leaf-occupancy assumptions

```
a=ceil(M/2) >= 2; ell=ceil(L/2) >= 1
h=number of edges on a root-to-leaf path
if h>=1:
    leaf_count >= 2*a^(h-1)
    n >= 2*ell*a^(h-1)
    h <= 1+log_a(n/(2*ell))
// Root-only trees h=0 cost one page, including the empty tree.
```

An internal root has at least two children; each later internal page has at least a. Depth h therefore has at least $2a^{h-1}$ leaves, each with at least ell records. Treat a leaf root, h=0, separately. A cold lookup reads h+1 pages, giving the general bound $O(1+\log_a(1+n/\ell))$. M≥3,L≥2 alone do not permit changing the logarithm base to B.

For any integer q≥3, choose M=3,$L=B=2^q$, exactly two children per internal page, and 2^q full B-record leaves. This legal tree has h=q and n=B^2. A cold lookup reads q+1 pages, while $\log_B n$ stays 2. The claimed uniform $O(\log_B n)$ bound fails because minimum internal fanout remains 2 instead of growing with B.

B+ height needs fanout and leaf-occupancy assumptions (continued)

To simplify, additionally require fixed-size keys/entries/records, one node per page, M=Θ(B), and L=Θ(B). Then $a,\ell=\Theta(B)$, so height is $O(1+\log_B n)$. A range with k outputs touches at most two partially used end leaves; every fully scanned interior leaf contributes at least ell outputs. Thus general index cost is $O(h+1+k/\ell)$, simplifying to $O(1+\log_B n+k/B)$ only under both capacity assumptions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Point-query and range-query I/O

Locating a leaf costs O(h+1) pages under the general height bound just proved. Index range cost is $O(h + 1 + k/\ell)$. Only M=Θ(B),L=Θ(B), fixed-size entries/records, and one node per page permit $O(1 + \log_B n + k/B)$.

Record pointers can add D data-page reads, worst-case D=k. Clustered fixed-size records at Θ(B) per page can instead bound those data pages by O(1+k/B). Producing k outputs still needs Ω(k) basic output work; CPU and page-I/O counts have different units.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Simplified B+ trees and database implementations

- PostgreSQL 18 documentation describes multilevel B-tree indexes whose pages form a doubly linked list at each level. Leaf tuples point to table rows and internal tuples to lower-level pages.
- This supports the practical motivation for multiway page indexes. It is not a guarantee that our simplified deletion steps reproduce PostgreSQL internals.
- Databases also need concurrency control, consistent reads during splits, WAL, and crash recovery. A single-threaded pointer diagram does not handle them.

References and attribution

Lin Cing, handwritten data structures notes. Pat Morin, Open Data Structures, § 14.2. PostgreSQL 18, B-tree implementation.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study